



Lecture –Introduction Object Oriented Programming (JAVA)

MR. MUHAMMAD SALEEM
LECTURER
DEPARTMENT OF COMPUTER SCIENCE

AGENDA

- Instructor Introduction
- Marks Distribution
- Recommended Books
- Who can learn this course

• RECAP!!

- What is Java?
- Why Java
- Platforms of Java



About Me

About me

Muhammad Saleem

Lecturer (Computer Science)

Qualification:

BS Computer Science University of Sindh, Jamshoro

MS Computer Science Sukkur IBA University

Masters in Education (M.Ed)

Marks distribution (3+1)

THEORY (03 CREDIT HOURS)

MID Term	30
Final Exam	50
Sessional Include	
• Assignments	5
• Project	10
• Quiz	5
Total	100

PRACTICAL (01 CREDIT HOUR)

Practical Exam	15
Project Viva	10
Practical Exam	10
Portfolio	15
Open Ended Lab-I	7.5
Open Ended Lab-II	7.5
Total	50

Who can learn this???

Anyone

- No experience in computer science or programming at all.
- Wants to develop his/her own programs using java
- Wants to be a powerful programmer

Beginners

- Computer Science students
- Self learners
- No experience in java
- Moving from other programming languages to java such as c/c++

Contents

Introduction to object oriented design, history and advantages of object oriented design, introduction to object oriented programming concepts, classes, objects, data encapsulation, constructors, destructors, access modifiers, const vs non-const functions, static data members & functions, function overloading, operator overloading, identification of classes and their relationships, composition, aggregation, inheritance, multiple inheritance, polymorphism, abstract classes and interfaces, generic programming concepts, function & class templates, standard template library, object streams, data and object serialization using object streams, exception handling.

My Contents

PROGRAMMING FUNDAMENTALS

- **JAVA**
- **JDK/JRE/JVM**
- **Installation**
- **IDE**
- **Variables**
- **Data types**
- **Operators**
- **Loops**
- **Arrays**
- **Math Functions**
- **String Manipulation**

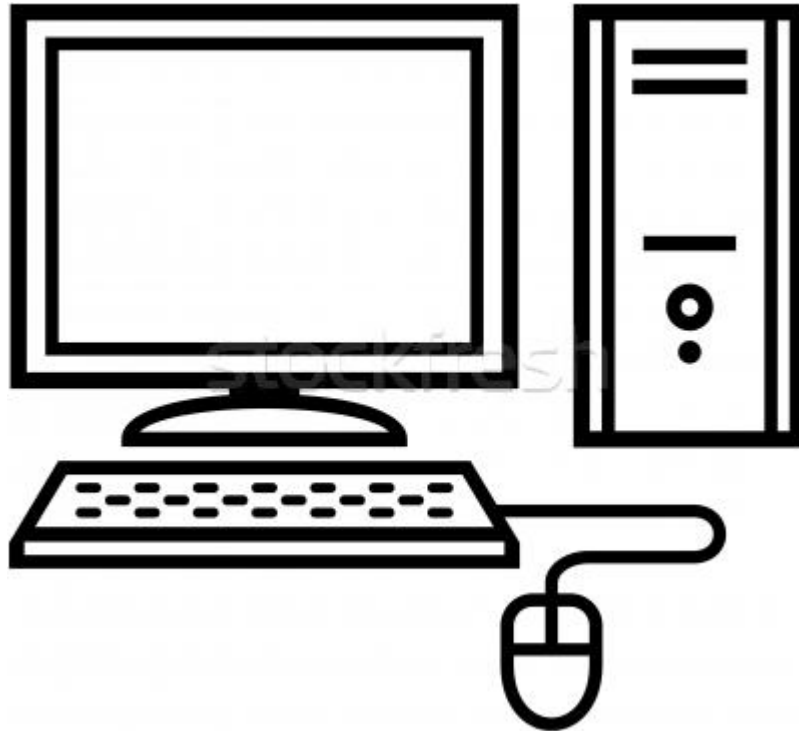
OBJECT ORIENTED PROGRAMMING

- **Classes**
- **Methods**
- **Object**
- **Constructor**
- **Access Modifiers**
- **This keyword**
- **Static Keyword**
- **Association**
- **Aggregation**
- **Inheritance**
- **IS-A Relationship**
- **HAS-A Relationship**
- **Super Keyword**
- **Encapsulation**
- **Accessor and Mutators**
- **Abstraction**
- **Polymorphism**
- **Abstract Class**

Don't be lazy go and grab
your Pen & Notebook



Computer Programming



Computer Programming

It is the process of giving instructions (commands) to the computer to do a meaningful task.

It is an act of teaching the computer on how to do a task.

The art and science of creating instructions for a computer to follow.

Creating a sequence of instructions to enable the computer to do something.

Why Programming?



Why Programming?

Computer is just a dumb machine made up of different electronic components. It is like a box which cannot do anything by itself.

It is the user who tells the computer “what it has to do?”

If we need our computer to perform some task, we first have to teach the computer in detail “how it will accomplish that task?”

Once the computer is taught about a particular task, it will completely obey it but cannot do anything that it is not taught to.

Why Programming?

Like the humans, we can teach the computer through communicating with it using a particular language.

The language that computer understands is *machine language*, also called as *binary language*. Machine language is the language of 0s and 1s.

We give detailed instructions to the computer to solve a particular task. *Programming* is the term that refers to teaching, instructing or giving commands to the computer.

Why Programming?

Programming is more about problem solving skills than writing the code itself.

Programming teaches you how to understand, analyze and solve the problems. It enhances your analytical reasoning abilities and helps you cope with daily real life problems as well.

Hence learning to program is important because it develops analytical and problem solving abilities.

Programmer

The person who gives the instructions (commands) to the computer is known as the *programmer*.

A person who designs and writes computer programs.



Instruction



Instruction

Instruction is any command given to the computer.

For example:

- 1) Add two variables A and B
- 2) Display result
- 3) Read file

Each of these is the individual instruction to the computer.

Program

NOW DO THIS!

NOW DO THIS!

NOW DO THIS!

NOW DO THIS!

NOW DO THIS!



Program

Program is a set (collection) of instruction to do a meaningful task.

A sequence of instructions that are interpreted and executed by a computer. It can be made of a single or hundred of instructions.

For example: In order to teach the computer on how to calculate average of three numbers? We need to give multiple instructions to the computer to do the task.

Program

Instruction 1: Get first number from the user and store it in A variable

Instruction 2: Get second number from the user and store it in B variable

Instruction 3: Get third number from the user and store it in C variable

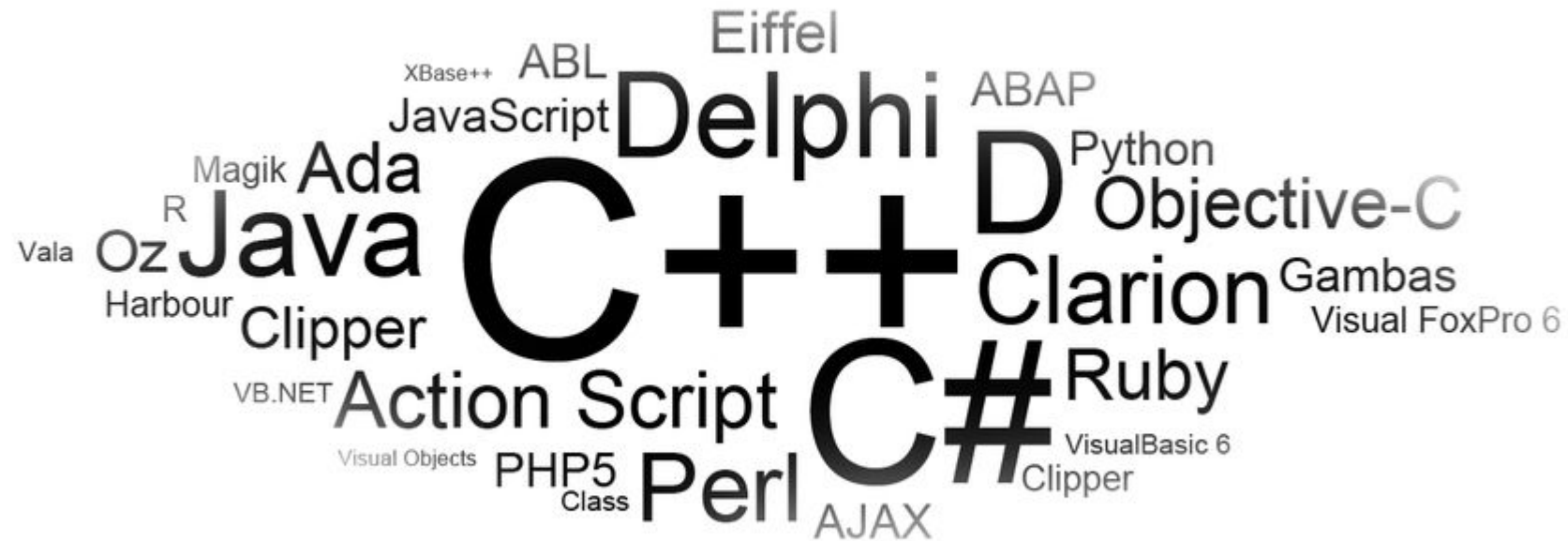
Instruction 4: Add A, B, C and store the result in SUM variable

Instruction 5: Divide SUM by 3 and store result in AVG variable

Instruction 6: Display AVG variable

Instructions 1-6 are used to solve a single task. This collection of instruction is known as a program.

Programming Language



Programming Language

A programming language is an artificial language designed to communicate instructions to a computer.

A programming language is a notation for writing programs.

A vocabulary and set of grammatical rules for instructing a computer to perform specific tasks.

Programming Language

Each language has a unique set of keywords (special words that it understands) and a special syntax (format) for organizing program instructions.

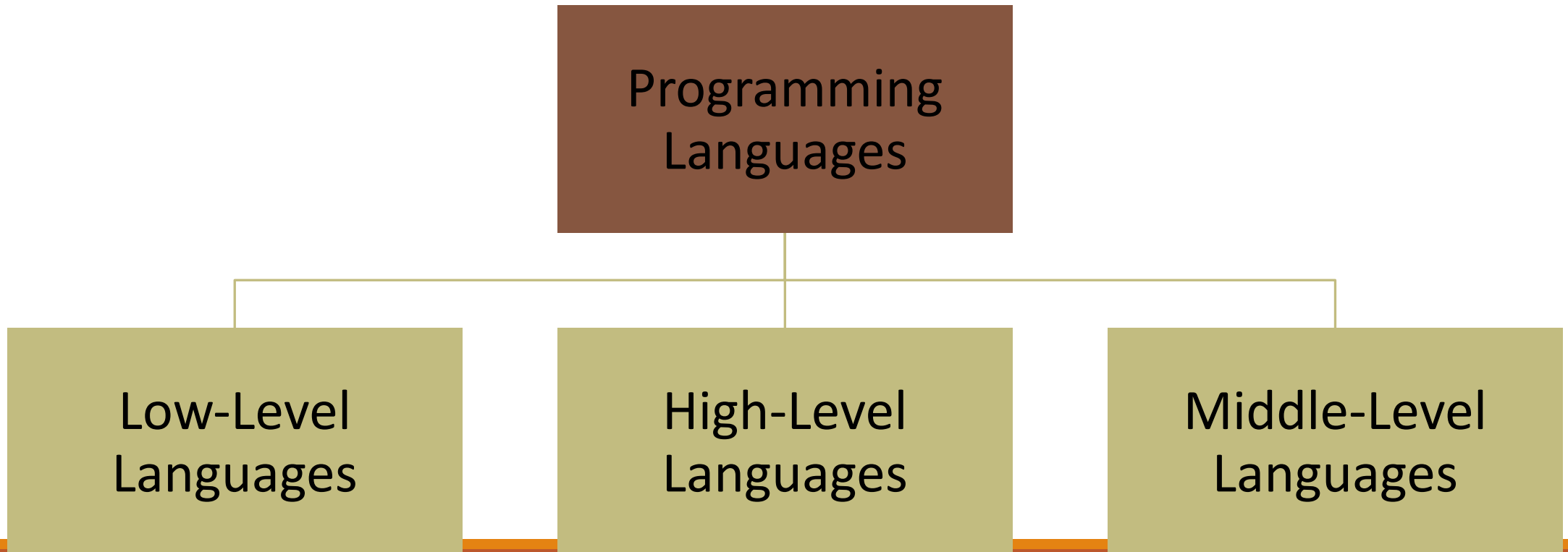
- **GW Basic**

There are many programming languages. For example:

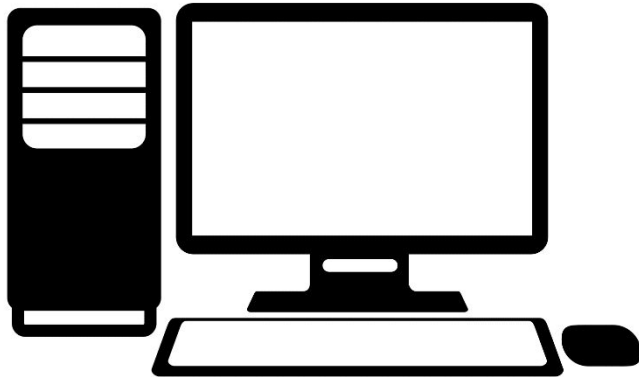
- **C**
- **C++**
- **JAVA**
- **Pascal**
- **COBOL**

Types of Programming Languages

There are three types of programming languages:



Low-Level Language



Machine (Computer)



Human (Programmer)



A language which is closer to machine (computer)

Low-Level Language

A low level language is one which is closer to the machine (computer).

It is easier for machines to understand and difficult for humans to understand.

It is faster in execution as compared to high and middle level languages.

Low-Level Language

Two of the types of low level languages are:

Machine
Language

Assembly
Language

Machine Language

It is one of the low level language.

It is the language of 0s and 1s.

Machine languages are the only languages directly understood by the computers.

While easily understood by computers, machine languages are almost impossible for humans to use because they consist entirely of numbers (0s and 1s).

Machine Language

It is the native language of the machines (computers).

Here all the instructions are written as code of binary sequence. For example:

In order to do addition, the code is: **10010001**

In order to decrement a number by one, the code is: **11011011**

In order to move data from one place to another, the code is: **10000111**

Machine Language

There are hundreds of instructions and each instruction has a binary code.

Is it possible to remember all the codes of hundreds of instruction?

Obviously not! Hence machine language almost impossible to understand.

Machine Language

Machine language program example:

10010010

11001010

01001010

11110101

00000101

00101000

11101010

10101010

Assembly Language

Assembly language is same as machine language but uses English like words to represent individual operations.

For example: Instead of binary codes it uses : **ADD, MOV, SUB, INC**

Assembly language is also a low-level language.

It is easier than the machine language but still it is very difficult to control a larger program using assembly.

Assembly Language

As assembly language contains English like words, which will not be understood by the computer (because it only understands 0s and 1s)

A translator first converts the assembly language program into machine language program.

Translator used with assembly language is called **Assembler**.

Assembly Language

Assembly language program example:

MVI A, 05h

MVI B, 9Ah

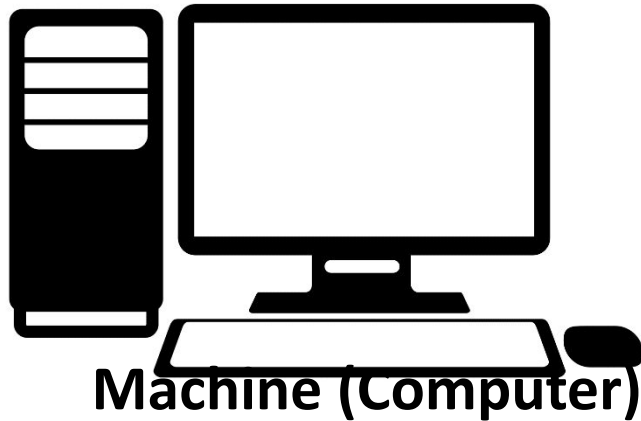
ADD B

INC A

STA 8200h

HLT

High-Level Language



Human (Programmer)



A language which is closer to human (programmer)

High-Level Language

A high level language is one which is closer to the human (programmer).

It is easier for humans to understand and difficult for machines to understand.

It is slower in execution as compared to low level languages.

High-Level Language

Like assembly language, it also uses English like words for the operations.

For example: **for, if, else, break, continue, while, include, using, import**

It is more easier than assembly language.

High-Level Language

- **GW Basic**

- **C++**

- **JAVA**

Some of the high level programming languages are:

- **Pascal**

- **COBOL**

- **Python**

- **C#**

- **Visual Basic**

- **J#**

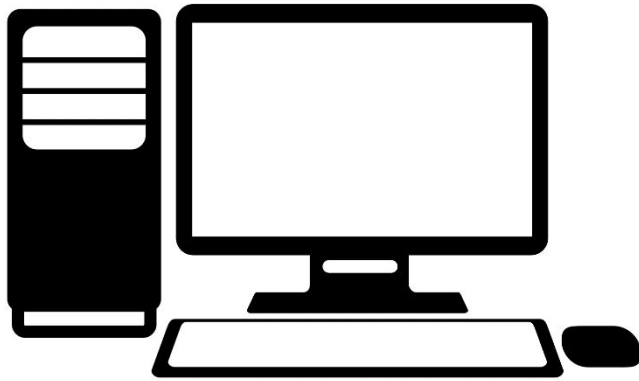
High-Level Language

High level language program example:

```
int main()
{
    int a = 5;
    int b = 6;

    if(a > b)
        cout<<"First number is greater.";
    else
        cout<<"Second number is greater.";
}
```

Middle-Level Language



Machine (Computer)



Human (Programmer)



A language which is some how closer to machine as well as human

Middle-Level Language

A middle level language is one which is closer to machine (computer) as well as to human (programmer).

A language that has the features of both low level and high level languages.

More formally, a high level language that allows you to write low level programs in it is called as middle level language.

Middle-Level Language

Some of the middle level programming languages are:

- **C**
- **IBM PL/S (Programming Language/Systems)**
- **BCPL (Basic Combined Programming Language)**
- **BLISS (Bill's Language for Implementing System Software)**

Source Code and Object Code

SOURCE CODE

The set of instructions written in any language other than machine language is called as source code.

It is not directly understood by the machine (computer).

OBJECT CODE

The set of instructions written in machine language is called as object code. It is also known as **machine code**.

It is the only code which is directly understood by the machine (computer).

Source Code and Object Code

SOURCE CODE

It is in the form of text.

It is human readable.

It is generated by human (programmer).

It is input to the language translator.

OBJECT CODE

It is in the form of binary numbers.

It is machine (computer) readable.

It is generated by the language translator.

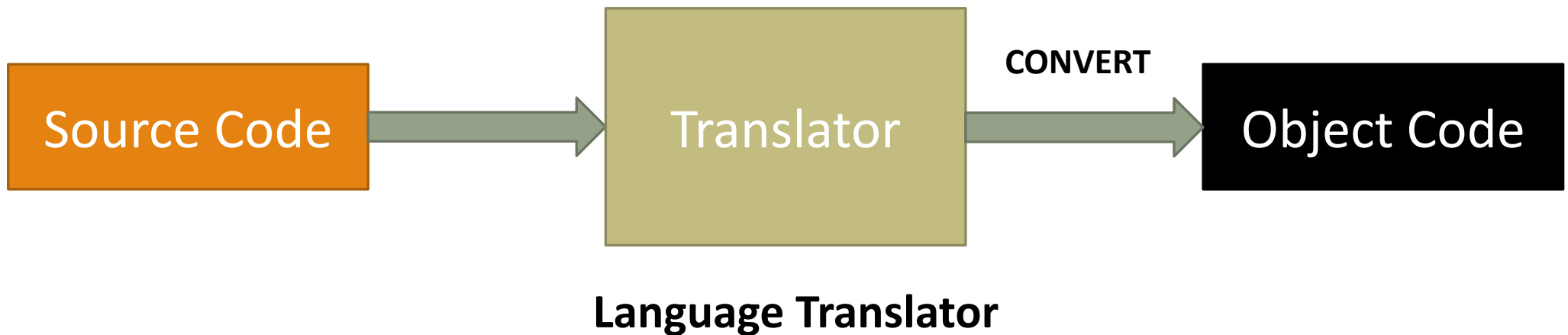
It is the output of the language translator.

Language Translators



Language Translators

Language translator is a program that converts the source code into the object code.



Why Language Translators?

Computer only understands object code (machine code).

It does not understand any source code.

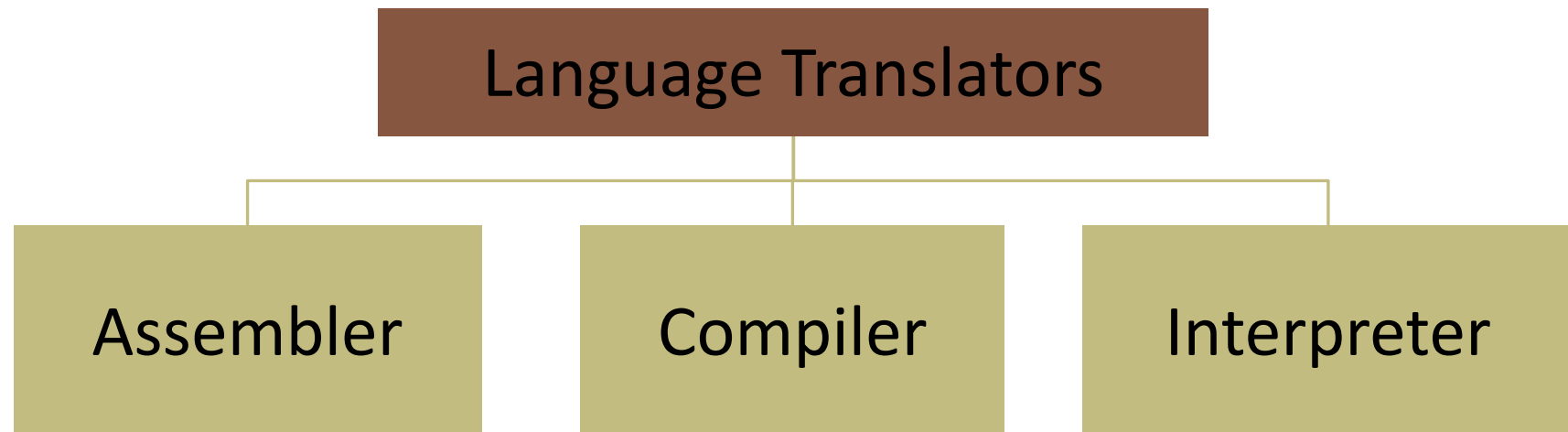
There must be a program that converts source code in to the object code so that the computer can understand it.

The language translator is one which does this job.

The programmer writes the source code and then translator converts it in machine readable format (object code).

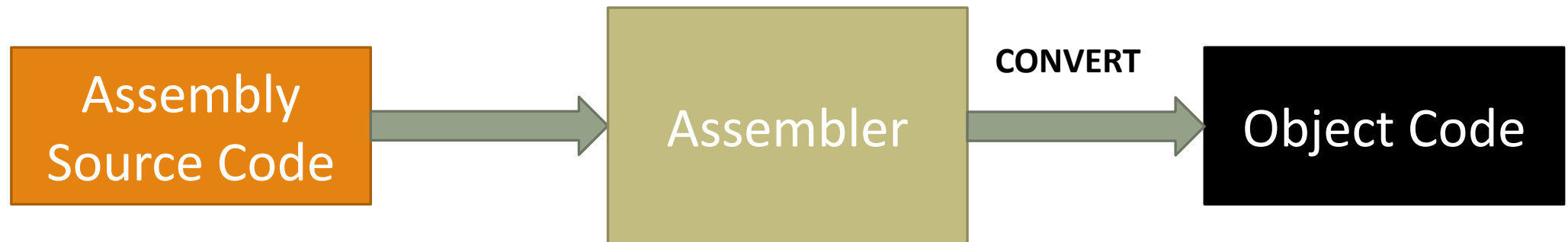
Types of Language Translators

There are three types of language translator:



Assembler

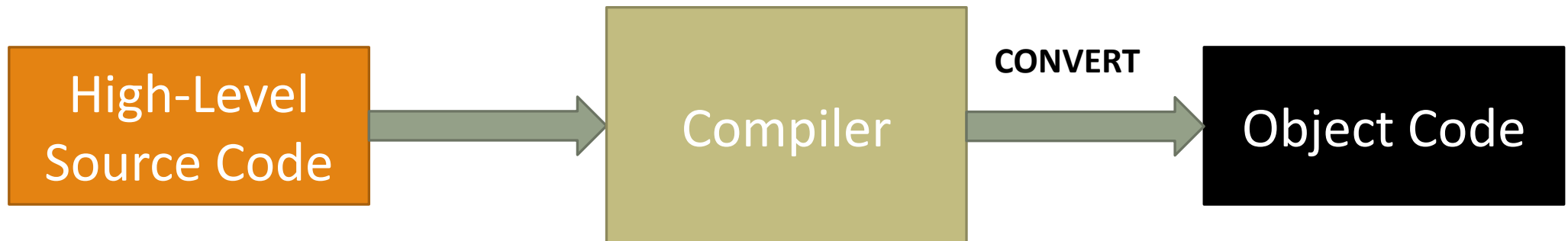
Assembler is the language translator that converts assembly language code into the object code (machine code).



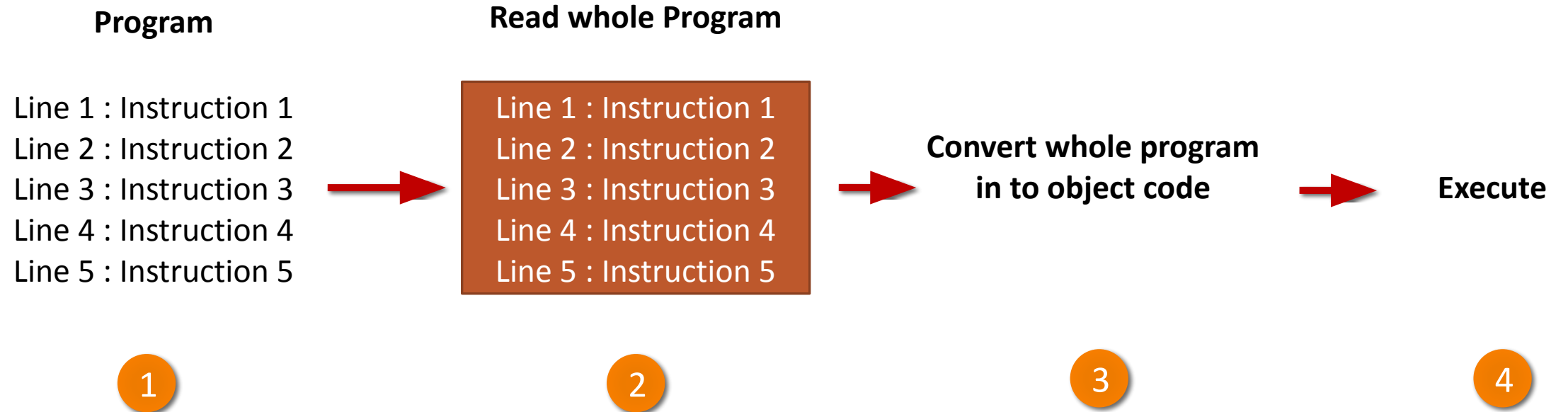
Compiler

Compiler is the language translator that converts high level language code in to the object code (machine code).

It converts the whole code at a time.



Compiler



Interpreter

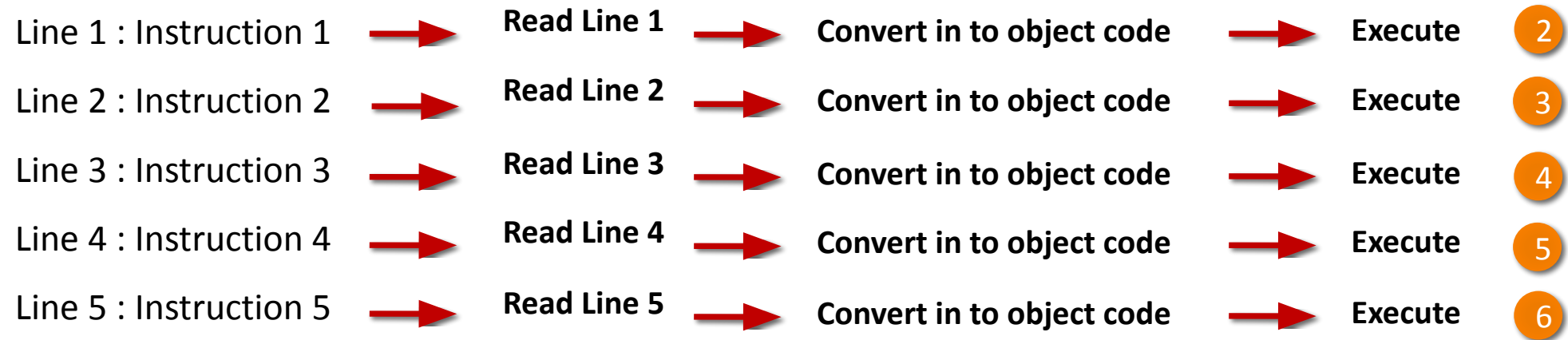
Interpreter is the language translator that converts high level language code in to the object code (machine code).

It converts the code line by line.



Interpreter

Program



1

Difference between Compiler and Interpreter

COMPILER

It converts whole code at a time.

It is faster.

Requires more memory.

Errors are displayed after entire program is checked.

Example: C, C++, JAVA.

INTERPRETER

It converts the code line by line.

It is slower.

Requires less memory.

Errors are displayed for every instruction interpreted (if any).

Example: GW BASIC, Ruby, Python

Bug



Bug

An error or defect occurred inside a computer program or hardware that causes it to produce an incorrect or unexpected result, or to behave in unintended ways is called as a **bug**.

Most of the bugs arise from mistakes and errors made by programmer in source code.

Bug

The term bug was used by Grace Hopper in 1946.

Hopper used to work on Mark II computer, there some error occurred in the system. The cause of the error was a moth (bug) trapped in a relay creating short circuit.

That caused the term bug to be coined.

Debugging



Debugging

It is the process of finding and fixing the bugs (errors) in the program.

It is the process of removing errors.

The programmer manually does this by examining the source code.

JAVA

What is Java?

Java is a **programming language** and a **platform**.

Java is a high level, robust, object-oriented and secure programming language.

Java was developed by *Sun Microsystems* (which is now the subsidiary of Oracle) in the year 1995.

James Gosling is known as the father of Java.

Before Java, its name was *Oak*. Since Oak was already a registered company, so James Gosling and his team changed the name from Oak to Java.

What is platform

- **Platform:** Any hardware or software environment in which a program runs, is known as a platform. Since Java has a runtime environment (JRE) and API, it is called a platform.
- There are 4 platforms or editions of Java:
 1. Java SE (Java Standard Edition): Develop applications that run on a desktop
 2. Java EE (Java Enterprise Edition): Develop server-side application
 3. Java ME (Java Micro Edition): Develop application for mobile devices.
 4. JavaFX: Develop distributed application
- **GO AND EXPLORE MORE ABOUT JAVA PLATFORMS A READING ASSIGNMENT**

Application Programming Interface (API)

- Also known as library
- Contains predefined java code that we can use to develop java programs
- Faster and easier development process
- No need to write anything from the scratch.

Integrated Development Environment (IDE)

- Write source code
- Compiler machine code
- Debug tool to find errors
- Build execute our program
- Development is faster and easier
- **Notepad, VSC, Netbeans, Eclipse, IntelliJ IDEA**

Why Java?

- The answer is that Java enables users to develop and deploy applications on the Internet for :
servers, desktop computers and small hand-held devices.
- The future of computing is being profoundly influenced by the Internet, and Java promises to remain a big part of that future.
- Java is the Internet programming language.
- Java is a general-purpose programming language.

Applications

According to Sun, 3 billion devices run Java. There are many devices where Java is currently used. Some of them are as follows:

1. Desktop Applications such as acrobat reader, media player, antivirus, etc.
2. Web Applications such as irctc.co.in, javatpoint.com, etc.
3. Enterprise Applications such as banking applications.
4. Mobile
5. Embedded System
6. Smart Card
7. Robotics
8. Games, etc.

What Else is Java?

According to Gosling:

- “An environment”
- “A platform”
- “A way of thinking”
- ...ok, whatever



History of Java

- The history of Java starts with the Green Team.
- Java team members (also known as Green Team).
- The principles for creating Java programming were "Simple, Robust, Portable, Platform-independent, Secured, High Performance, Multithreaded, Architecture Neutral, Object-Oriented, Interpreted, and Dynamic".
- Java was developed by James Gosling, who is known as the father of Java, in 1995.
- James Gosling and his team members started the project in the early '90s.



JAVA'S HISTORY

WHY JAVA WAS RENAMED OAK?

- **Why Oak?** Oak is a symbol of strength and chosen as a national tree of many countries like the U.S.A., France, Germany, Romania, etc.
- 6) In 1995, Oak was renamed as "**Java**" because it was already a trademark by Oak Technologies.

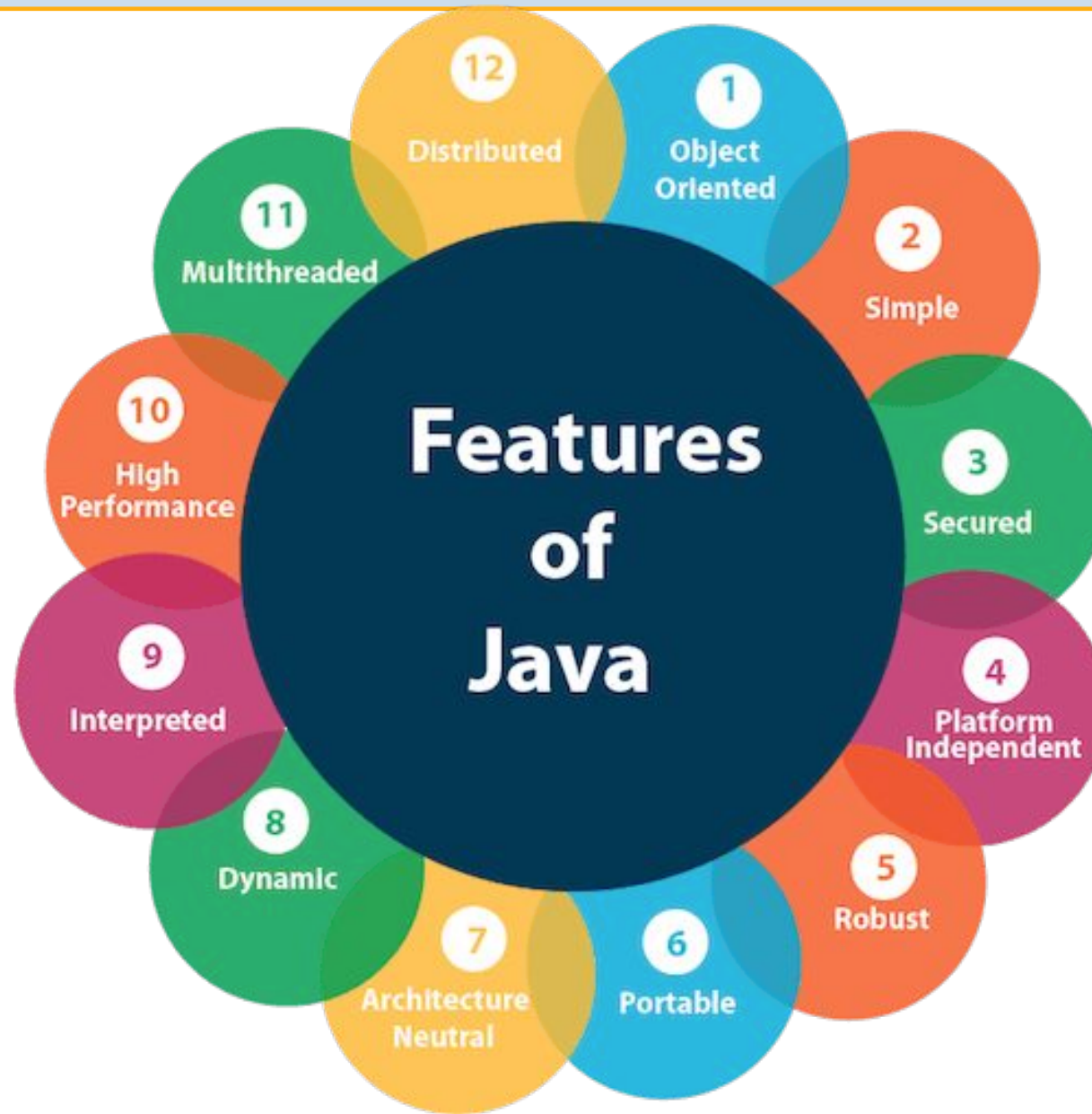


Java Version History

JDK Alpha and Beta (1995)	JDK 1.0 (23rd Jan 1996)	JDK 1.1 (19th Feb 1997)	J2SE 1.2 (8th Dec 1998)	J2SE 1.3 (8th May 2000)	J2SE 1.4 (6th Feb 2002)
J2SE 5.0 (30th Sep 2004)	Java SE 6 (11th Dec 2006)	Java SE 7 (28th July 2011)	Java SE 8 (18th Mar 2014)	Java SE 9 (21st Sep 2017)	Java SE 10 (20th Mar 2018)
Java SE 11 (September 2018)	Java SE 12 (March 2019)	Java SE 13 (September 2019)	Java SE 14 (Mar 2020)	Java SE 15 (September 2020)	Java SE 16 (Mar 2021)
		Java SE 17 (September 2021)	Java SE 18 (to be released by March 2022)		

FEATURES OF JAVA

- ❑ The primary objective of **Java programming** language creation was to make it portable, simple and secure programming language.
- ❑ Apart from this, there are also some excellent features which play an important role in the popularity of this language.
- ❑ The features of Java are also known as Java buzzwords.



Simple:

- Java is very easy to learn, and its syntax is simple, clean and easy to understand.
- According to Sun Microsystems, Java language is a simple programming language because:

Object-oriented:

- Java is an **object-oriented** programming language.
- Everything in Java is an object.
- Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behavior.

Platform Independent:

- Java is platform independent because it is different from other languages like C, C++, etc.
- which are compiled into platform specific machines while Java is a write once, run anywhere language.
- A platform is the hardware or software environment in which a program runs.

Secured

- Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:
 - **No explicit pointer**
 - **Java Programs run inside a virtual machine sandbox**

Robust

- The English meaning of Robust is strong. Java is robust because:
- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.

Architecture-neutral

- ❑ Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.
- ❑ In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture.
- ❑ However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

Portable

- ❑ Java is portable because it facilitates you to carry the Java bytecode to any platform.
- ❑ It doesn't require any implementation.

JVM

JVM (Java Virtual Machine) is an abstract machine.

It is called a virtual machine because it doesn't physically exist.

It is a specification that provides a runtime environment in which Java bytecode can be executed.

The JVM performs the following main tasks:

- Loads code
- Verifies code
- Executes code
- Provides runtime environment

JRE

JRE is an acronym for Java Runtime Environment

The Java Runtime Environment is a set of software tools which are used for developing Java applications.

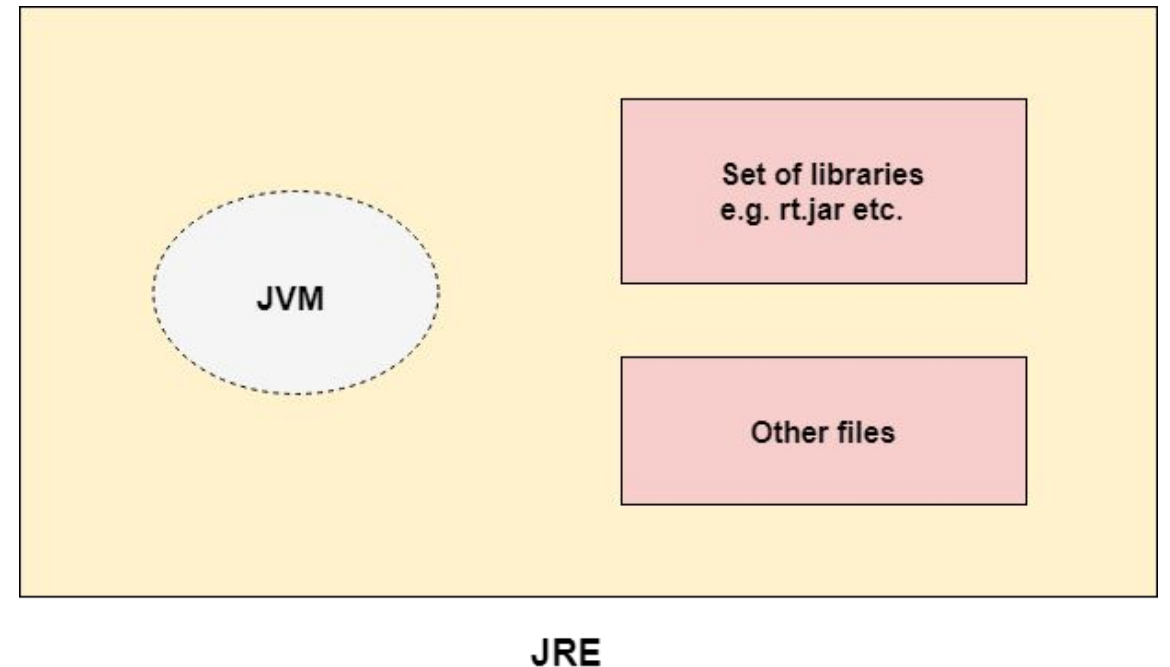
It is used to provide the runtime environment.

It is the implementation of JVM.

It physically exists.

It contains a set of libraries + other files that JVM uses at runtime.

The implementation of JVM is also actively released by other companies besides Sun Micro Systems.



JDK



JDK is an acronym for Java Development Kit.

The Java Development Kit (JDK) is a software development environment which is used to develop Java applications and **applets**.

It physically exists.

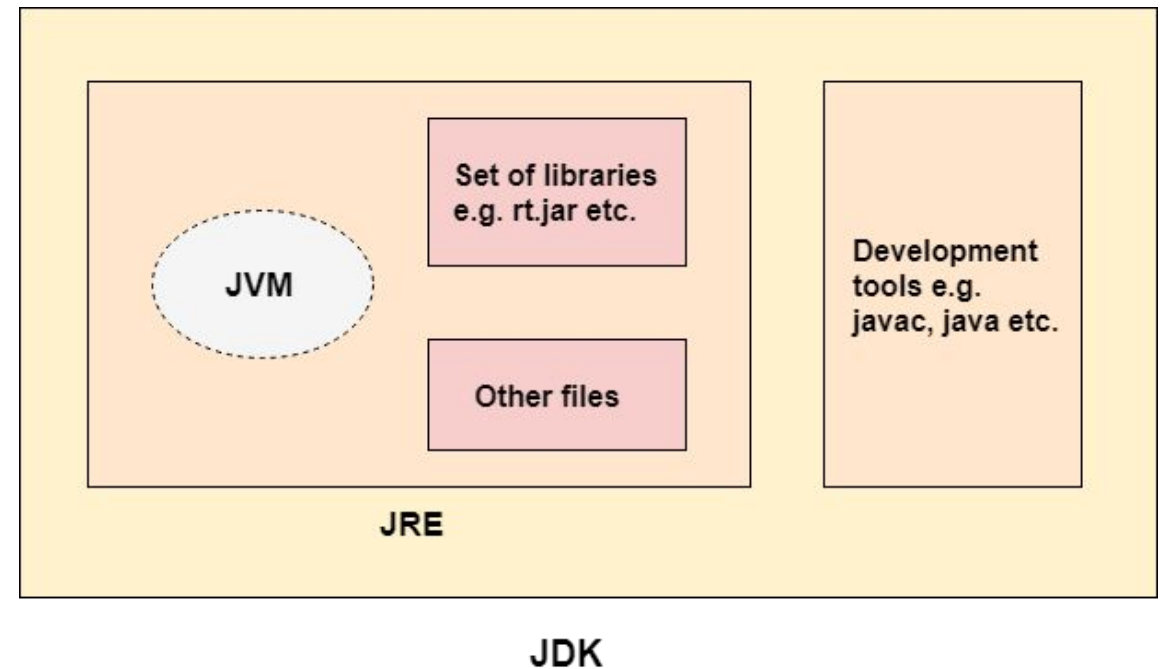
It contains JRE + development tools.

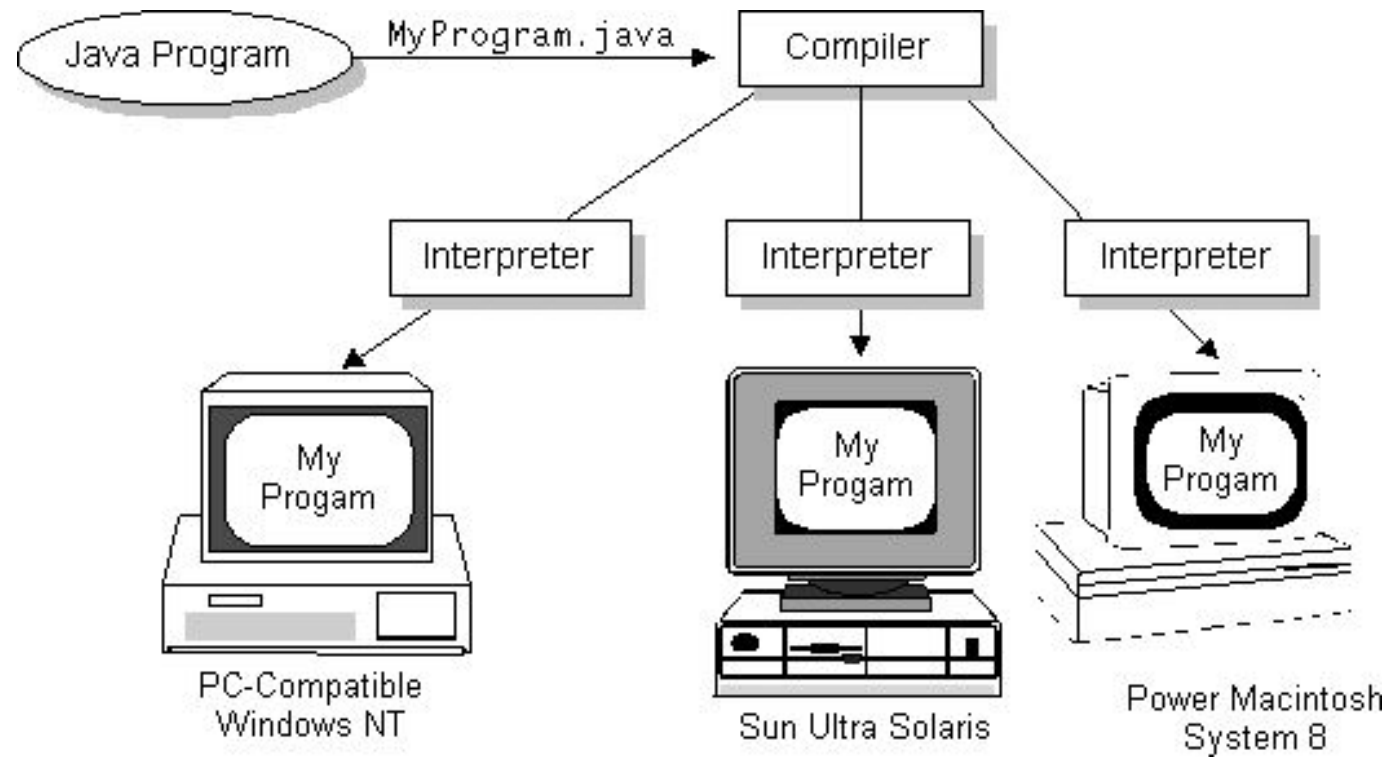
JDK is an implementation of any one of the below given Java Platforms released by Oracle Corporation:

- Standard Edition Java Platform
- Enterprise Edition Java Platform
- Micro Edition Java Platform

JDK

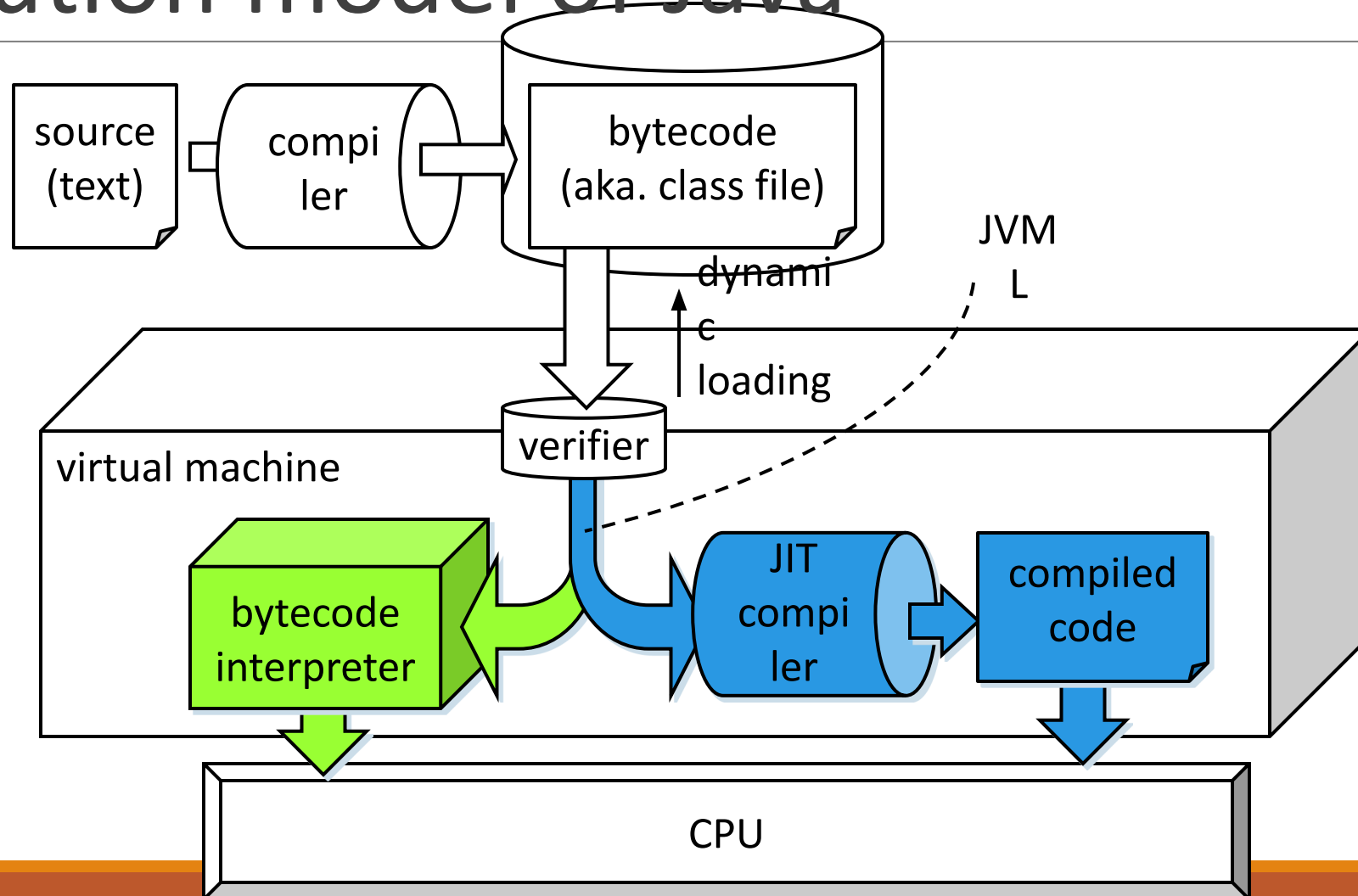
The JDK contains a private Java Virtual Machine (JVM) and a few other resources such as an interpreter/loader (java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc), etc. to complete the development of a Java Application.

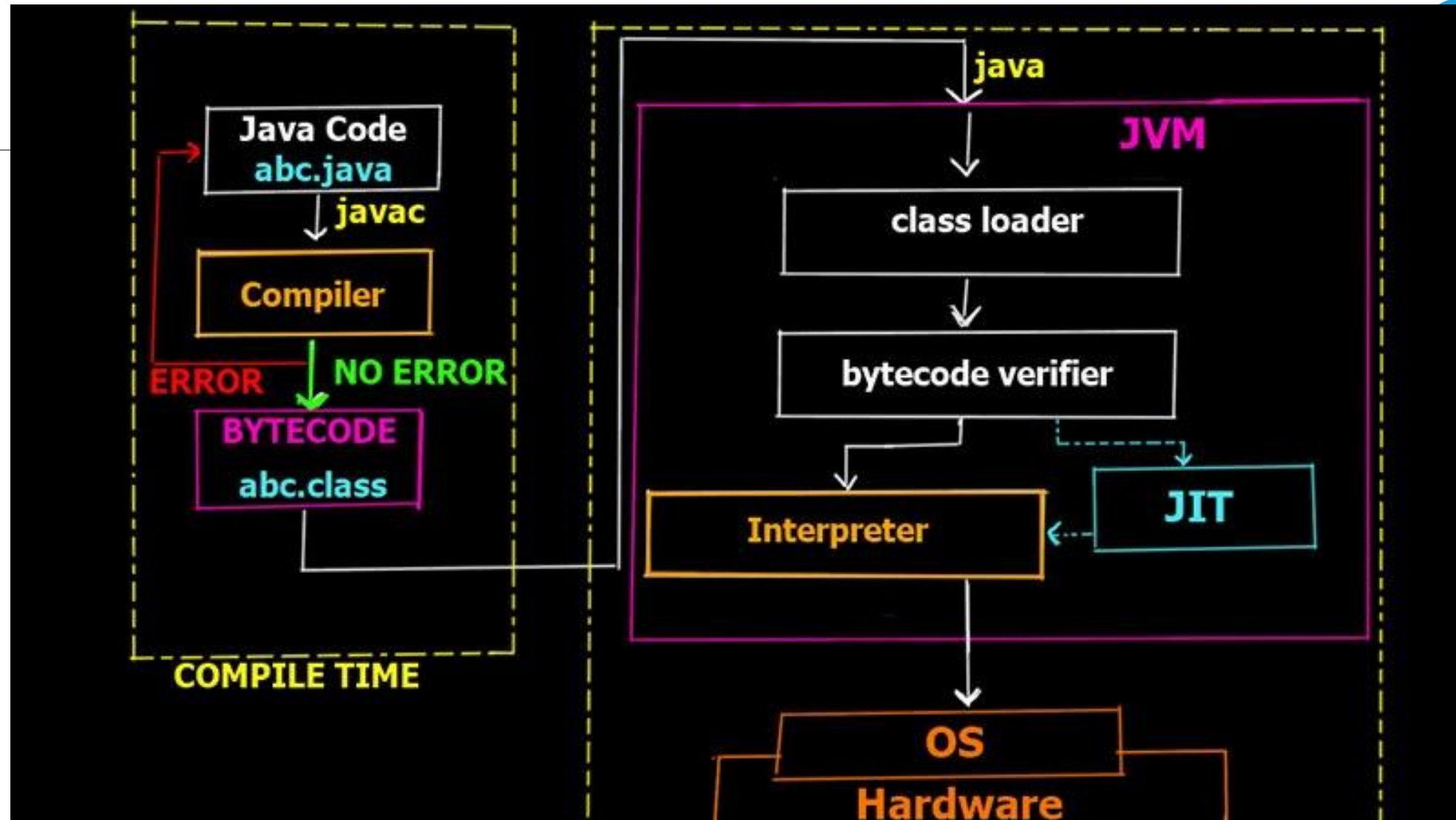




Write Once, Run Anywhere

execution model of Java





The JIT

Just-In-Time compiler

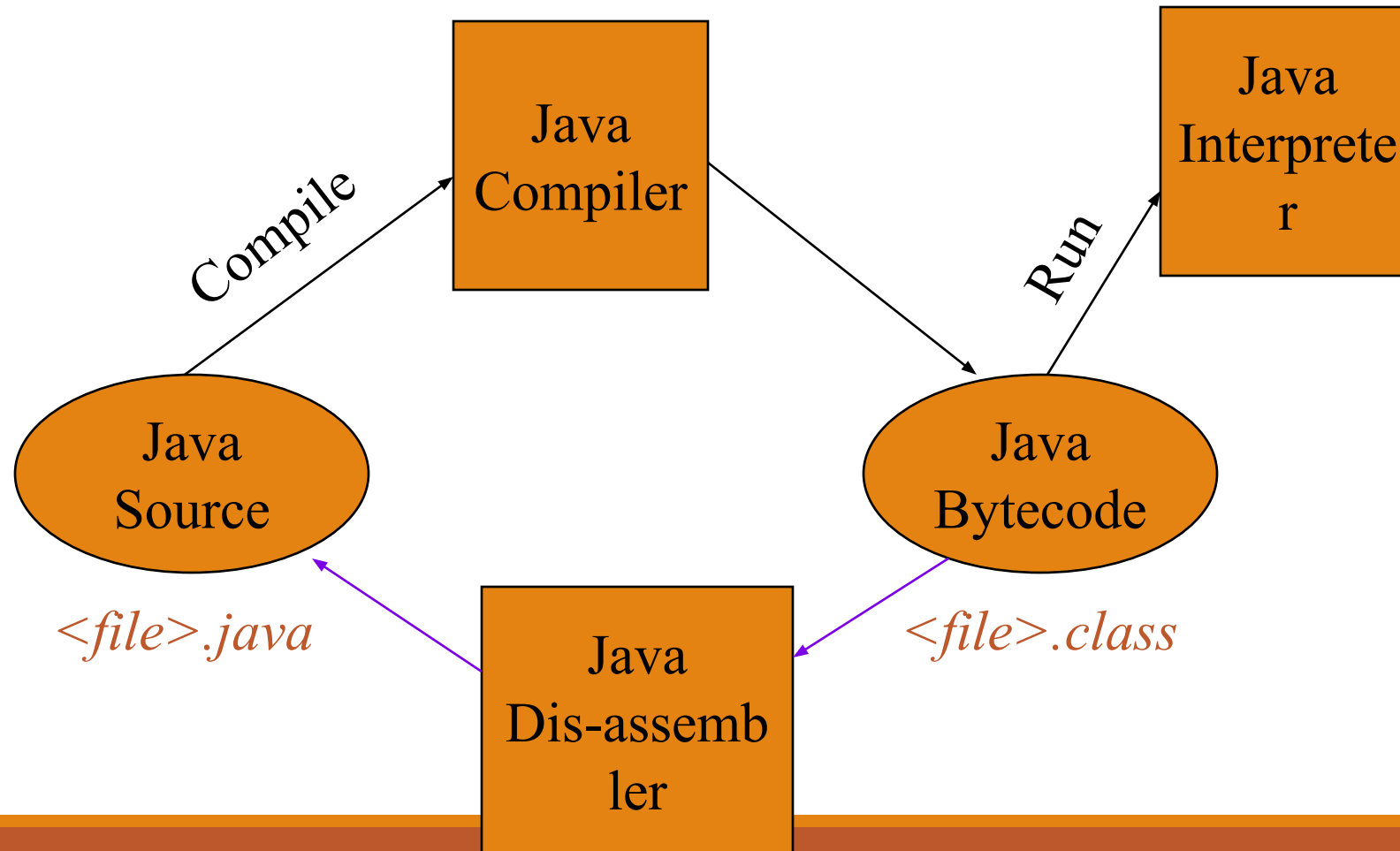
Translates bytecode into machine code at runtime

- Performance increase 10-30 times

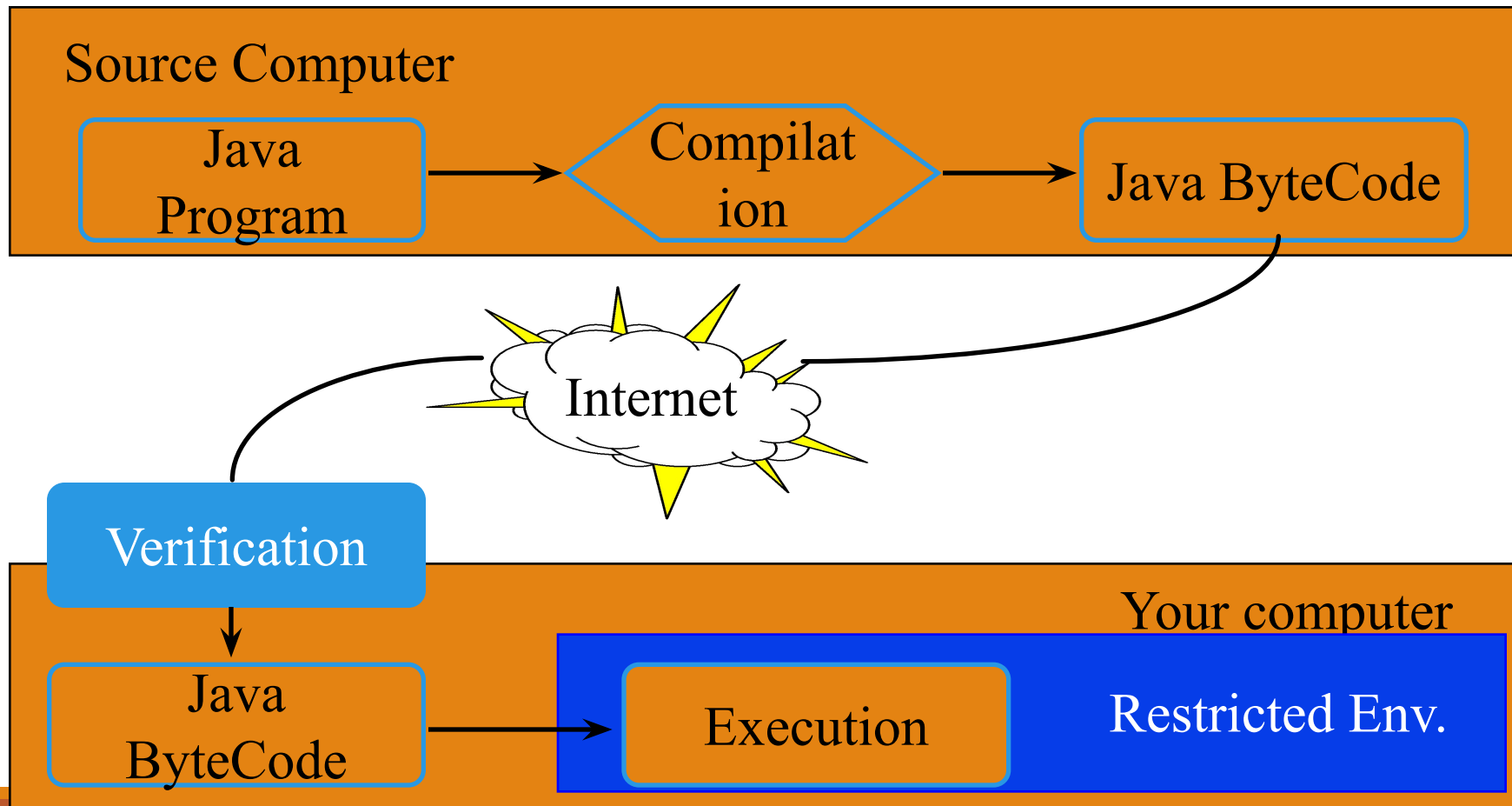
Now the default for most JVM's

- Can be turned off if desired
- JIT can apply statistical optimizations based on runtime usage profile

Java Developer's Kit (II)



Prepare and Execute Java



Java Developer's Kit (JDK)

JDK or the Java Development Kit is a set of:

a JRE as subset.

a Java compiler,

a Java interpreter,

developer tools,

Java API libraries,

documentation which can be used by Java developers to develop Java-based applications.

additional files needed to write Java applets and applications.

Java Runtime Environment (JRE)

JRE or the Java Runtime Environment is a minimum set that includes :

Java interpreter,

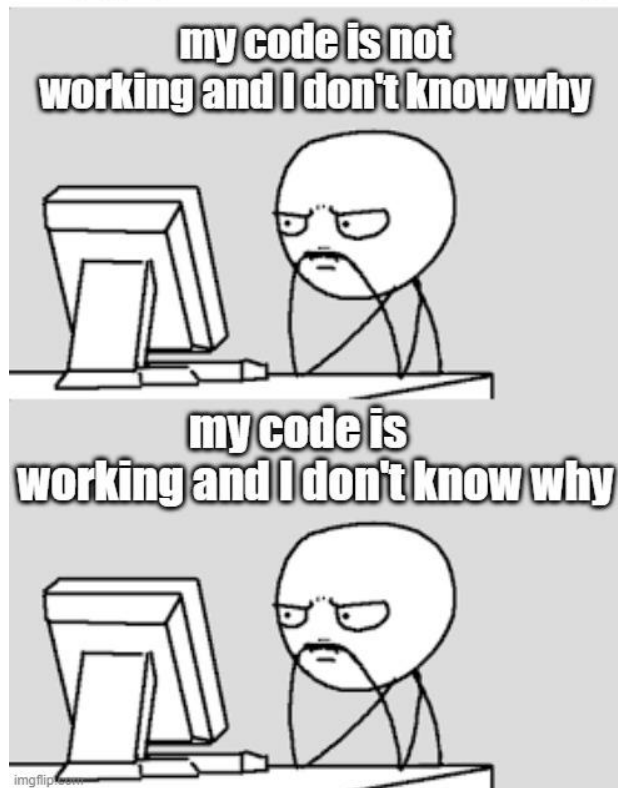
Java API libraries,

Java browser plug-in,

- which make up the minimum environment to execute Java-based applications.

Other additional components to run applications and applets written in Java.

**TWO THINGS
DEVELOPERS COULD BE
THINKING ABOUT WHEN
YOU SEE THEM LIKE THIS**



Thanks 🥰